

09 — Codex Skills: installation and use

The question

How do I find, install, and use a Codex skill on Windows, and how can it help improve a scientific figure without changing the calculation?

This guide uses Windows, VS Code, Codex, Anaconda, Python, and Jupyter Notebook. The running example is K-Dense-AI's third-party [scientific-visualization skill](#)^[1]. Instructions were checked on **September 20, 2026**. Interfaces and discovery paths can change, so the distinctions below matter when revisiting this guide.

This chapter is documentation, not an installation record. Writing it does not install the skill, change a Python environment, or execute the plotting exercise. Installation commands below are for a future session when you decide to install it.

1. What a skill changes

A skill gives Codex a reusable procedure for a particular kind of task. Think of a laboratory protocol: it can explain what to inspect, which resources to consult, what steps to follow, and how to check the result. It does not train a new model or guarantee that the output is correct.

For our example, a useful procedure is: inspect the existing plotting code and intended publication size, preserve the data, improve visual presentation, export suitable formats, and inspect the exports. Your prompt supplies the specific file and constraints; the skill supplies reusable guidance.

Item	What it provides	Example
Ordinary prompt	Instructions for the current request	"Make this legend readable and keep every data point."
Skill	A reusable workflow with optional supporting files	scientific-visualization
Plugin	An installable package that can distribute skills and integrations	A package containing several related workflows
MCP	Model Context Protocol: a way to connect an assistant to tools and data	A server exposing document search
Python library	Code executed by the selected Python interpreter	Matplotlib draws the figure; NumPy stores arrays

A skill can tell Codex to use Matplotlib, but copying a skill folder does not install Matplotlib. An MCP connection may provide a tool, but it does not by itself specify a complete plotting workflow. Plugin packaging is another distribution route; this chapter teaches installation of one standalone skill folder. See the official [customization overview](#)^[2], [skills and plugins](#)^[3], and [MCP guide](#)^[4].

Codex initially sees a skill's name and description, then reads its instructions when selected. A matching task can trigger it implicitly, or you can select it explicitly with \$ or /skills in the Codex interface. These are Codex commands, not PowerShell commands. See [Build skills](#)^[5].

2. What is inside a skill folder?

The essential file is `SKILL.md`. The inspected K-Dense example contains these 18 files:

```
scientific-visualization/  
  SKILL.md  
  references/  
    color_palettes.md  
    journal_requirements.md  
    matplotlib_examples.md  
    publication_guidelines.md  
    sources.md  
  scripts/  
    _common.py  
    export_plan.py  
    figure_export.py  
    image_metadata.py  
    palette_audit.py  
    style_presets.py  
    style_preview.py  
  assets/  
    color_palettes.py  
    nature.mplstyle  
    presentation.mplstyle  
    publication.mplstyle  
    publisher_profiles.json
```

Component	How to read it
SKILL.md	The entry point: metadata, intended use, and workflow instructions
references/	Detailed material to consult when relevant, such as figure design guidance
scripts/	Programs or helpers; inspect their inputs, outputs, and imports before running them
assets/	Files used by the workflow, such as templates and style presets
agents/openai.yaml	Optional interface metadata, invocation policy, and declared tool dependencies

The top of a `SKILL.md` usually contains YAML metadata between two `---` lines. This teaching example is not a replacement for the upstream file:

```
---  
name: scientific-visualization  
description: Improve scientific figures and publication exports.  
---
```

The `name` identifies the skill. The `description` helps Codex judge relevance. Instructions below the metadata describe the work. Merely naming a folder correctly is insufficient if its `SKILL.md` is missing or malformed.

The inspected [skill directory](#)^[1] does not contain `agents/openai.yaml`, `requirements.txt`, a `pyproject.toml`, or a dependency lockfile. Its metadata reports version 1.2 and Python 3.11+ compatibility; these are the skill's declarations, not a complete tested environment specification. Keep the folder together: helpers use sibling files and `../assets`. Copying only `SKILL.md` breaks those relationships. Installation also does not turn helpers such as `style_presets.py` into an importable library in every notebook.

3. Where to find skills and what to check

Start with official OpenAI examples and the system `skill-installer`; third-party GitHub repositories provide additional workflows. An author's repository is useful evidence about that author's code, but it is not an OpenAI endorsement. The official [OpenAI skills repository](#)^[6] also points readers toward current plugin examples.

For a candidate, check these items before running its helpers:

- 1. **Identity:** confirm the repository owner, exact folder, and `SKILL.md` name. Similar names can describe different skills.
- 2. **Scope:** read the description and workflow. Does it address your figure type? Does it assume a different assistant or operating system?
- 3. **Dependencies:** inspect imports and shell commands. Distinguish essential packages from optional examples and unrelated repository tools.
- 4. **Effects:** look for files written, external requests, credentials, and commands that install software or change configuration.
- 5. **Version and license:** inspect recent changes and the license. Record a commit or release when reproducibility matters.

The K-Dense repository has an [MIT license](#)^[7]. Installing a single subfolder does not copy that root license. Keep a source and version record, and preserve applicable notices when redistributing files. The skill also asks users to cite its accompanying paper when used in scientific work; read such behavioral instructions before adopting third-party workflows.

There are three separate dependency layers:

Layer	What is needed
Reading the skill	Codex and the skill files; no plotting package is required just to read instructions
Downloading the skill	The installer needs Python and network access; its Git fallback also needs Git
Running a figure workflow	Packages imported by the selected code, such as NumPy and Matplotlib; additional helpers may need more

In this version, palette auditing and export planning mostly use the standard library. Image metadata inspection needs Pillow for raster files and `pypdf` for PDF files; SVG metadata uses the standard library. Plotly static export is a different workflow involving Kaleido and Chrome. Ordinary Matplotlib SVG/PDF export does not require those tools or TeX. See the [helper scripts](#)^[8].

Do not install every dependency from the entire scientific-skills repository for one plotting task. Its broader Python, WSL, and uv setup instructions are not automatically requirements of this single skill. Start with the existing Anaconda environment and inspect the exact helper you intend to use. No account, API key, MCP server, or additional skill was found to be mandatory for this local plotting example.

4. Choose the installation scope and path

Current official documentation uses .agents/skills for standalone user and repository skills. The skill-installer bundled on the machine used to prepare this guide instead defaults to \$CODEX_HOME/skills, normally ~/.codex/skills. These are different facts: the documented discovery location and a particular installed helper's default destination. Do not assume that every version scans every historical location.

Scope	Windows example	Practical effect
Personal, current documented location	C:\Users\Hainan\.agents\skills\scientific-visualization\	Available across this user's projects in the same host environment
Project, current documented location	<repository>\.agents\skills\scientific-visualization\	Travels with this repository if deliberately committed
Observed bundled installer default	C:\Users\Hainan\.codex\skills\scientific-visualization\	Actual default of the inspected helper; verify recognition in your client

Repository discovery scans relevant .agents/skills directories from the current working directory up to the repository root. Same-name skills are not merged. Prefer one intentional installation to several competing copies. The authoritative discovery reference is [Build skills](#)^[5].

In PowerShell, \$env:USERPROFILE usually identifies your Windows user folder. \$env:CODEX_HOME, if set, changes the Codex home used by the inspected installer; it does not make .codex and .agents interchangeable. A remote, container, or WSL session has its own filesystem and home folder.

5. Install from GitHub, step by step

5.1 Open the right input area

For natural-language instructions, use the **Codex chat box** in VS Code. For commands labelled **PowerShell**, choose **Terminal** → **New Terminal** and ensure that the terminal profile is PowerShell. Do not type the displayed ps c:\...> prompt itself. [Chapter 04](#) explains the current working directory.

5.2 Beginner route: ask skill-installer

Codex chat box — a future installation request:

```
$skill-installer
Install only scientific-visualization from:
https://github.com/K-Dense-AI/scientific-agent-skills/tree/main/skills/scientific-visualization

First inspect the source, its dependencies, and the paths
supported by my installed Codex version.
Use a personal installation in my user .agents/skills folder.
Pass that parent folder explicitly as --dest if needed.
Report the exact destination and every file operation.
Do not install Python packages or change my conda environments.
If an existing copy is present, report it without overwriting.
```

For a project installation, replace the personal-scope instruction with: “Use .agents/skills under the repository root as the explicit destination.” Codex may need approval to write outside the project or download files, depending on your current permission settings.

5.3 Explicit PowerShell route

The commands below use the locally available installer. Its location can differ on another machine. If `Test-Path` reports `False`, ask Codex to locate its installed `skill-installer` rather than downloading random replacement scripts.

PowerShell — inspect paths; no skill is installed yet:

```
Get-Location
python -c "import sys; print(sys.executable)"
$skillCodexHome = $env:CODEX_HOME
if (-not $skillCodexHome) {
    $skillCodexHome = Join-Path $env:USERPROFILE '.codex'
}
$installer = Join-Path $skillCodexHome (
    'skills\.system\skill-installer\scripts\' +
    'install-skill-from-github.py'
)
Test-Path -LiteralPath $installer
python -B $installer --help
```

If python is unavailable, use the existing Anaconda PowerShell Prompt or the known full path to your Python executable. Selecting an interpreter in VS Code does not prove an already-open terminal uses it; check `sys.executable` as above.

PowerShell — choose one destination:

```
# Personal installation:
$skillParent = Join-Path $env:USERPROFILE '.agents\skills'
```

For a project installation, open the repository folder in VS Code, verify its root, then use this alternative assignment:

```
$projectRoot = git rev-parse --show-toplevel
if ($LASTEXITCODE -ne 0) { throw 'Open the intended Git repository.' }
$skillParent = Join-Path $projectRoot '.agents\skills'
```

`--dest` receives the **parent skills directory**. The installer adds `scientific-visualization` beneath it. Giving the final skill directory as `--dest` would create an unnecessary nested directory.

PowerShell — installation command; downloads and writes files:

```
$installArgs = @(
    '--repo', 'K-Dense-AI/scientific-agent-skills',
    '--path', 'skills/scientific-visualization',
    '--ref', 'main',
    '--dest', $skillParent
)
python -B $installer @installArgs
if ($LASTEXITCODE -ne 0) { throw 'Skill installation failed.' }
```

If you change `$skillParent`, rebuild `$installArgs` before rerunning: an existing array retains the old destination value. The command is an alternative to the chat-box route; do not run both to create duplicate installations.

The helper also accepts `--url` with the GitHub folder URL. In the inspected implementation, a URL containing `/tree/main/` supplies its own reference; adding `--ref` does not override that `main`. For a pinned version, use `--repo` plus `--path` and replace `main` with a real reviewed commit or tag. Do not copy an invented commit identifier.

5.4 What does installation change?

For the inspected installer, installation copies the selected skill directory and its files into the destination. It creates missing destination parents. The download route temporarily downloads the repository ZIP, then copies only the requested skill. Temporary work normally lives under `%TEMP%\codex\skill-install-*`; the actual location follows Python's temporary-directory setting, and cleanup is attempted afterwards. An existing destination is an error, not an automatic update. The `-B` flag above prevents Python bytecode cache files in the installer's own directory.

This file-copy operation does not itself run the downloaded helper scripts, install Python packages, change conda environments, modify the plotting script, edit `config.toml` or `AGENTS.md`, or register an MCP server. Project installation adds files that Git may show as untracked; it does not commit or push them. Temporary cleanup may be incomplete after an interrupted process. These details were checked against the local installer and can be rechecked in the [installer source](#)^[9].

6. Confirm recognition and use the skill

First verify the files. Continue in the same PowerShell session, or set `$skillParent` again if you opened a new terminal:

```
$skillFolder = Join-Path $skillParent 'scientific-visualization'
Test-Path -LiteralPath (Join-Path $skillFolder 'SKILL.md')
Get-ChildItem -LiteralPath $skillFolder
```

Then type `$` in the **Codex chat box** and look for the skill, or use `/skills` if available in your interface. Newly installed skills should be discovered automatically; if the list remains stale, start a new Codex conversation or restart the extension. Saving a skill file is not proof that the current session has selected it.

The inspected installer's `list-skills.py` checks its default `$CODEX_HOME/skills` directory for installed annotations. A skill deliberately placed in `.agents/skills` may therefore lack that annotation. Check Codex's actual skill selector and loaded path instead of treating the installer's list as the discovery result.

Codex chat box — verify without changing your project:

```
$scientific-visualization
Read this skill and report the SKILL.md path you loaded.
Summarize how it would review a Matplotlib figure.
List any dependencies needed for the proposed workflow.
Do not edit files, install packages, or run plotting scripts.
```

For implicit use, a request such as “Improve this Matplotlib figure for a journal submission” may match the description. For a learning exercise, use the explicit name and ask for the loaded path. Availability, selection, and successful execution are three different checks.

Symptom	What to inspect
Skill missing from the selector	Destination, SKILL.md, metadata, active workspace, and client version; then refresh
Available but not selected automatically	Task-description match and policy.allow_implicit_invocation in agents/openai.yaml, if present; try explicit invocation
Two similar skills appear	Duplicate names across personal, project, or plugin locations; inspect actual paths
Skill is selected but plotting fails	Python interpreter, imports, fonts, data paths, and write permissions
Installer reports an existing destination	Treat it as an update decision; inspect and back up the existing copy
Works in a terminal but fails in a notebook	The notebook may use a different Python kernel
Works on Windows but is missing remotely	Installations belong to the filesystem where Codex is running

Jupyter Notebook — run in a Python code cell:

```
import sys
print(sys.executable)
import numpy
import matplotlib
print(numpy.__version__, matplotlib.__version__)
```

Compare the interpreter path with the terminal check. Select the intended notebook kernel if they differ. If imports fail, identify the correct environment before considering package installation; the skill folder cannot fix a missing library. See the [VS Code notebook guide](#) ^[10].

7. Update, disable, or uninstall

7.1 Update deliberately

An installed skill does not automatically follow its upstream GitHub source. Pulling this handbook does not update a personal installation; a project copy tracked by this repository can change when pulled commits change it. The installer has no overwrite/update switch in the inspected version.

1. Identify the exact active skill path and record its source and current version.

2. Copy your current folder, including local edits, to a backup **outside every scanned skills directory**.
3. Download the reviewed new version into a separate staging folder using `--dest`.
4. Compare the old and new `SKILL.md`, scripts, and dependencies. After review, move the old active folder outside the discovery locations and put the reviewed new folder at its original path. Replacing the folder, rather than merging its contents, avoids retaining obsolete helper files.
5. Refresh Codex and repeat the read-only recognition check. Keep the backup until your normal workflow succeeds.

Do not leave an “old” renamed folder containing `SKILL.md` under a scanned directory: it may still be discovered. Updating a skill does not require updating every Python package.

7.2 Disable without deleting

The official Build skills page shows a `[[skills.config]]` entry pointing to `SKILL.md`. The configuration reference describes the path as a **skill directory**. Because these current descriptions differ, use the documented example first and verify the result in your installed version; do not claim disabling worked merely because the file was saved.

TOML configuration file — edit the effective Codex `config.toml`, not the terminal:

```
[[skills.config]]
path = "C:/Users/Hainan/.agents/skills/scientific-visualization/SKILL.md"
enabled = false
```

Use your actual installed path. The usual configuration location is `C:\Users\Hainan\.codex\config.toml`; a custom `CODEX_HOME` can change it. Preserve existing settings and edit an existing matching entry instead of repeatedly appending duplicates. Restart Codex and verify the skill is unavailable. If the installed version expects a directory path, follow its schema and verify again. See [Build skills](#)^[5] and [configuration reference](#)^[11].

A reversible alternative is moving the entire skill folder outside all discovery locations, then refreshing Codex. This is different from disabling only automatic invocation: a skill can remain explicitly callable even when its implicit-invocation policy is off.

7.3 Uninstall

Locate the same verified folder in File Explorer, back up any local edits, and remove **only that skill folder**. Remove its obsolete matching configuration entry if you created one. Refresh Codex and confirm it is absent. Check for another copy if it still appears. Do not delete the entire `.agents` or `.codex` directory.

Uninstalling the skill leaves your plots, Python packages, and conda environments in place. Removing a project skill is a repository change to review with `git status`; it is not automatically committed.

8. Practical example: improve a figure without changing the science

The existing [plotting example](#) compares the local PCG64 generator with the global MT19937 generator. It imports data-generation functions from [example 01](#). Both use seed 5, but they produce different streams. Preserving the seed alone is insufficient: preserve the generator type, draw order, parameters, and resulting arrays as well.

8.1 A reusable request

Codex chat box — use after the skill is installed:

```
$scientific-visualization
Improve the scientific presentation of:
examples/02_plot_generated_data.py

Read the skill and report its loaded path.
Inspect the script and its imported data-generation functions.
Keep those functions, both generator types (PCG64/MT19937),
seed 5, draw order, N_TRAIN=10, and NOISE_STD=4*0.03 unchanged.
Preserve exp(-x/2), the 300-point curve grid, and every array.
Do not smooth, filter, resample, refit, or remove observations.

Improve only fonts, colors, markers, legend placement, spacing,
and export settings. Keep titles and labels scientifically true.
Use a readable font available on this Windows machine.
Use color plus marker or line distinctions for grayscale use.
Keep a clear two-panel comparison at about 7 inches wide.
Avoid clipped labels and legends overlapping observations.

Preserve the existing PNG output and its destination.
Additionally export:
docs/images/09-generated-data-styled.svg
docs/images/09-generated-data-styled.pdf
Do not install packages or modify the Python environment.

Before editing, capture the arrays used in each panel and curve.
After editing, compare old and new values, shapes, and dtypes.
Require exact equality for this deterministic styling-only task.
Check the exported figures visually at the intended print size.
Report edits, output paths, comparisons, and any unavailable checks.
Preserve unrelated work; do not commit or push.
```

This prompt is a future exercise, not a claim that the figure has already been changed or exported. If you want to preserve the PNG's appearance too, ask Codex to create a separate plotting script instead of changing the existing script.

8.2 What the plotting changes can look like

The following is a small **Python fragment**, to adapt inside the plotting code after a figure named `fig` exists and before `plt.close(fig)`. It is not a complete program. `output_dir` is deliberately explicit because a notebook's working directory may differ from the script directory:

```
from pathlib import Path

# Replace this path with your intended existing output directory.
output_dir = Path("docs/images")
if not output_dir.is_dir():
    raise FileNotFoundError("Check the working directory first.")

for ax in fig.axes:
    ax.tick_params(labelsize=9)
    ax.xaxis.label.set_size(10)
    ax.yaxis.label.set_size(10)
    ax.title.set_size(11)

for suffix in ("svg", "pdf"):
    target = output_dir / f"09-generated-data-styled.{suffix}"
    fig.savefig(target, bbox_inches="tight", facecolor="white")
```

Matplotlib determines the output format from the extension here. SVG and PDF support vector elements, but raster images inside a figure remain raster. Increasing DPI does not make text larger. Inspect at the final physical size and check the publication's font and export requirements. `bbox_inches="tight"` can change the final bounding dimensions, so omit it when exact physical dimensions must be preserved. See [Matplotlib savefig](#) ^[12].

For editable SVG text, `svg.fonttype="none"` retains text but depends on fonts available to the viewer; `"path"` preserves glyph outlines instead. For PDF, `pdf.fonttype=42` is a possible TrueType setting. Choose deliberately for the publisher's requirements. If changing to `layout="constrained"`, remove conflicting `tight_layout()` calls. See [Matplotlib fonts](#) ^[13] and [constrained layout](#) ^[14].

8.3 Check the result

- Compare the actual old and new arrays with `numpy.testing.assert_array_equal`; matching shapes alone are insufficient. Also check dtypes explicitly. Comparing a new array only with its own copy does not test the edit.
- Open both exports and inspect labels, legends, panel spacing, missing glyphs, and grayscale distinctions. A file existing on disk is only the first check.
- Review the code diff: changes should concern presentation and exports. Explain any inability to reproduce the original environment before attributing numerical differences to styling.
- In a notebook, restart the kernel and run relevant cells in order when checking reproducibility; stale variables can hide a changed calculation.

The expected outcome is the same scientific data with clearer presentation and two additional export files. The exercise passes only after both numerical and visual checks succeed.

9. Prompt archive: the request behind this chapter

The following is an English translation of the original request, retained for reuse. Its instruction to provide a reply first records the original request; the later instruction to create chapter 09 authorized the handbook files. Neither request authorized installing the example skill.

Codex chat box — guide-authoring prompt:

Please write a beginner's guide to installing and using Codex Skills, for learning and future reference.

I use Windows + VS Code + Codex, as well as Anaconda, Python, and Jupyter Notebook. I am learning these tools. Explain plainly and concretely; do not assume I know the CLI.

Use scientific-visualization as a running example and explain:

1. What a skill is, how it affects Codex, and how it differs from prompts, plugins, MCP, and Python libraries.
2. The roles of SKILL.md, references, scripts, and assets.
3. Where to find skills and check sources, scope, dependencies.
4. Personal versus project installation, locations, GitHub installation with skill-installer, and all file changes.
5. Recognition, explicit and automatic invocation, and troubleshooting when the skill does not take effect.
6. Updating, disabling, and uninstalling.
7. A practical Matplotlib request to improve fonts, colors, legends, and layout while preserving calculated results and exporting SVG/PDF.

Example source:

<https://github.com/K-Dense-AI/scientific-agent-skills/tree/main/skills/scientific-visualization>

Check current official documentation, especially differences between .agents/skills and .codex/skills across versions. Label commands as Codex chat-box input or PowerShell input.

Explain in Chinese with English technical terms.

Use personal-handbook Markdown, steps, comparison tables, and copyable examples suitable for US Letter printing.

First provide the complete introduction in the reply.

Only write the tutorial: do not actually install a skill or modify environments or project files.

Also include this prompt in the document and assign the next number in the handbook sequence.

The subsequent instruction was to create 09-main-...md directly. This chapter, its bilingual companion, and their print editions implement that later instruction.

References

1. <https://github.com/K-Dense-AI/scientific-agent-skills/tree/main/skills/scientific-visualization>
2. <https://learn.chatgpt.com/docs/customization/overview>
3. <https://learn.chatgpt.com/docs/skills-and-plugins>
4. <https://learn.chatgpt.com/docs/extend/mcp>
5. <https://learn.chatgpt.com/docs/build-skills>
6. <https://github.com/openai/skills>
7. <https://github.com/K-Dense-AI/scientific-agent-skills/blob/main/LICENSE.md>
8. <https://github.com/K-Dense-AI/scientific-agent-skills/tree/main/skills/scientific-visualization/scripts>
9. <https://github.com/openai/skills/blob/main/skills/.system/skill-installer/scripts/install-skill-from-github.py>
10. <https://code.visualstudio.com/docs/datascience/jupyter-notebooks>
11. <https://learn.chatgpt.com/docs/config-file/config-reference>
12. https://matplotlib.org/stable/api/_as_gen/matplotlib.figure.Figure.savefig.html

13. <https://matplotlib.org/stable/users/explain/text/fonts.html>
14. https://matplotlib.org/stable/users/explain/axes/constrainedlayout_guide.html